Лабораторна робота №5

«Управління станом застосунку з використанням StatefulWidget»

Мета

Ознайомитися з поняттям стану (state) у Flutter-застосунку, навчитися створювати StatefulWidget, змінювати стан за допомогою setState(), реалізовувати взаємодію з користувачем та оновлення інтерфейсу відповідно до змін даних.

Теоретичні відомості

1. Поняття стану у Flutter

Стан (state) — це набір даних, які можуть змінюватися під час роботи застосунку та впливають на відображення інтерфейсу.

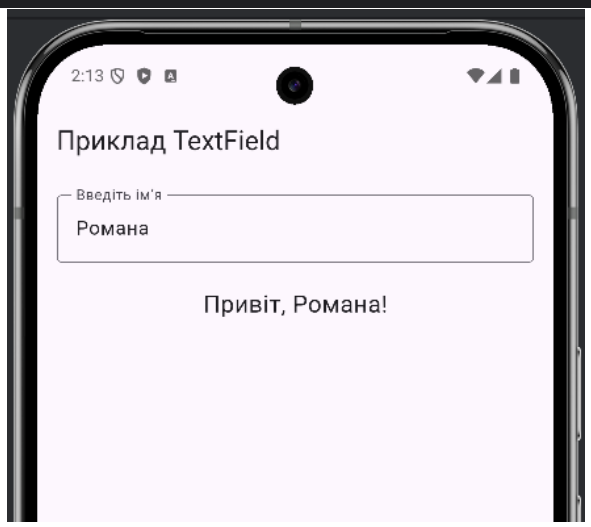
Приклади стану:

- лічильник кліків;
- вибраний елемент списку;
- введений текст:

```
String userName = '';

Column(
 children: [
   TextField(
     decoration: const InputDecoration(
       labelText: 'Введіть ім'я',
       border: OutlineInputBorder(),
     ),
     onChanged: (value) {
       setState(() {
         userName = value;
       });
     },
   ),
```

```
  const SizedBox(height: 20),
  Text(
    'Привіт, $userName!',
    style: const TextStyle(fontSize: 20),
  ),
 ],
);
```



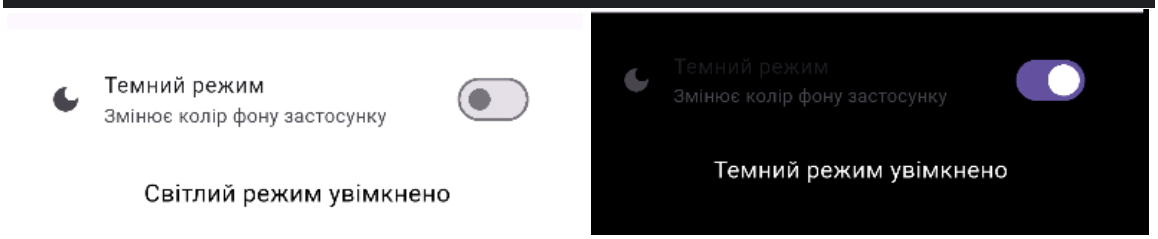
Пояснення:

- String userName — змінна стану.
- onChanged — викликається при кожному введенні символу.
- setState() — оновлює інтерфейс.
- Текст привітання автоматично змінюється.

- перемикач (Switch):

```
bool isDarkMode = false;
Column(
 children: [
   SwitchListTile(
     title: const Text('Темний режим'),
     subtitle: const Text('Змінює колір фону застосунку'),
     value: isDarkMode,
     onChanged: (value) {
       setState(() {
         isDarkMode = value;
```

```
            });
          },
          secondary: const Icon(Icons.dark_mode),
        ),
        const SizedBox(height: 20),
        Text(
          isDarkMode
              ? 'Темний режим увімкнено'
              : 'Світлий режим увімкнено',
          style: TextStyle(
            fontSize: 18,
            color: isDarkMode ? Colors.white : Colors.black,
          ),
        ),
      ],
    );
```



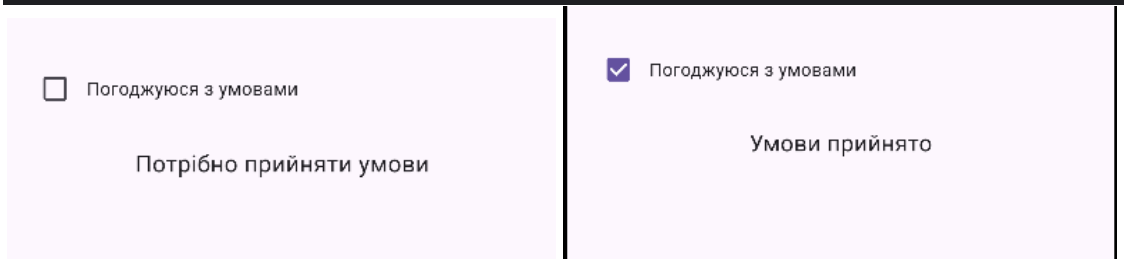
Пояснення:

- bool isDarkMode — змінна стану.
- value — поточне значення перемикача.
- onChanged — спрацьовує при зміні.
- setState() — оновлює інтерфейс.
- secondary — іконка зліва.

- прапорець (Checkbox):

```
bool isChecked = false;
Column(
 children: [
   Row(
     children: [
```

```
      Checkbox(
        value: isChecked,
        onChanged: (value) {
          setState(() {
            isChecked = value ?? false;
          });
        },
      ),
      const Text('Погоджуюся з умовами'),
    ],
  ),
  const SizedBox(height: 20),
  Text(
    isChecked
        ? 'Умови прийнято'
        : 'Потрібно прийняти умови',
    style: const TextStyle(fontSize: 18),
  ),
],
),
```



Пояснення:

- bool isChecked — зберігає стан прапорця.
- value — поточне значення.
- onChanged — спрацьовує при зміні.
- value ?? false — захист від null.
- Текст статусу змінюється відповідно до стану.

2. Метод setState()

setState() повідомляє Flutter, що дані змінилися і потрібно перебудувати інтерфейс.

⚠ Важливо:

- Викликати тільки при зміні даних.
- Не виконувати довгі обчислення всередині setState().

Приклад setState() для відображення лічильник натискань на кнопку:

```
class MyWidget extends StatefulWidget {
 const MyWidget({super.key});
 @override
 State<MyWidget> createState() => _MyWidgetState();
}
class _MyWidgetState extends State<MyWidget> {
 int counter = 0;
 @override
 Widget build(BuildContext context) {
   return Scaffold(
     body: Center(
       child: Text('$counter'),
     ),
     floatingActionButton: FloatingActionButton(
       onPressed: () {
         setState(() {
           counter++;
         });
       },
       child: Icon(Icons.add),
```

```
      ),
    );
  }
}
```

3. Життєвий цикл StatefulWidget

Основні методи:

- initState() — виконується один раз при створенні
- build() — будує інтерфейс
- dispose() — викликається перед видаленням віджета

Завдання на лабораторну роботу

1. Продовжити роботу над проєктом, розпочатим у Лабораторній роботі №4.
2. Екран повинен бути реалізований як StatefulWidget.
3. Повинен бути створений окремий клас стану (_ScreenNameState).
4. Всі змінні стану повинні зберігатися всередині класу State.
5. Оновлення інтерфейсу повинно виконуватись виключно через setState().
6. Обов’язкові змінні стану. У кожному варіанті мають бути використані мінімум:
    - bool — для Switch / Checkbox / обраного стану
    - int — лічильник дії
    - String — значення з TextField
7. Обов’язкові інтерактивні елементи:
    1. Іконка “обране”

- Змінює колір при натисканні.
- Реалізується через bool.

2. Кнопка дії
    - Збільшує лічильник.
    - Відображає поточне значення на екрані.
3. Checkbox або Switch

   Змінює відображення елементів інтерфейсу:
    - показ/приховування тексту
    - зміна стилю
    - поява бейджа
    - зміна кольору
4. TextField
    - Зчитує введений текст.
    - Відображає результат нижче.
    - Використовує onChanged.

Контрольні запитання

1. Що таке стан (state) у Flutter-застосунку?
2. Яке призначення методу createState()?
3. Для чого використовується метод setState()?
4. Що відбувається після виклику setState()?
5. Чому не рекомендується виконувати довгі обчислення всередині setState()?
6. Які методи життєвого циклу має StatefulWidget?
7. Як реалізувати оновлення тексту під час введення даних у TextField?
8. Як зробити кнопку неактивною залежно від стану змінної?

9. Які типи даних найчастіше використовуються для збереження стану?
10. Що станеться, якщо змінити змінну стану без виклику setState()?
11. У яких випадках доцільно використовувати StatefulWidget?